

Homework Assignment 2

1. Compute the response time and turnaround time when running five jobs of length 200 with the FIFO and SJF schedulers.

Output:

- **FIFO Scheduler**

```
tejaljadhav — jadhav2@pceasapp961:~/HW2 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
#####
CU Denver | Anschutz
#####
CU Denver | Anschutz Medical Campus network - authorized users only!
#####
Reminders:
# Users of the CU Denver | Anschutz Medical Campus network are subject to all local, state, and
# federal laws and regulations as well as CU Denver | Anschutz Medical Campus policies while
# using CU Denver | Anschutz Medical Campus resources. Unauthorized access is strictly prohibited.
# Any unauthorized access or tampering with this equipment or network will result in civil and/or
# criminal prosecution.
#####
jadhav2@csegrid.ucdenver.pvt's password:
[jadhav2@csci-gnode-03 ~]$ cd HW2
[jadhav2@csci-gnode-03 HW2]$ ls
scheduler.py
[jadhav2@csci-gnode-03 HW2]$ python3 scheduler.py -p FIFO -l 200,200,200,200,200 -c
ARG policy FIFO
ARG jlist 200,200,200,200,200

Here is the job list, with the run time of each job:
Job 0 ( length = 200.0 )
Job 1 ( length = 200.0 )
Job 2 ( length = 200.0 )
Job 3 ( length = 200.0 )
Job 4 ( length = 200.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 200.00 secs ( DONE at 200.00 )
[ time 200 ] Run job 1 for 200.00 secs ( DONE at 400.00 )
[ time 400 ] Run job 2 for 200.00 secs ( DONE at 600.00 )
[ time 600 ] Run job 3 for 200.00 secs ( DONE at 800.00 )
[ time 800 ] Run job 4 for 200.00 secs ( DONE at 1000.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 200.00 Wait 0.00
Job 1 -- Response: 200.00 Turnaround 400.00 Wait 200.00
Job 2 -- Response: 400.00 Turnaround 600.00 Wait 400.00
Job 3 -- Response: 600.00 Turnaround 800.00 Wait 600.00
Job 4 -- Response: 800.00 Turnaround 1000.00 Wait 800.00
Average -- Response: 400.00 Turnaround 600.00 Wait 400.00
[jadhav2@csci-gnode-03 HW2]$
```

- **SJF Scheduler**

```
tejaljadhav — jadhav2@pceasapp961:~/HW2 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[jadhav2@csci-gnode-03 HW2]$
[jadhav2@csci-gnode-03 HW2]$
[jadhav2@csci-gnode-03 HW2]$ python3 scheduler.py -p SJF -l 200,200,200,200,200 -c
ARG policy SJF
ARG jlist 200,200,200,200,200

Here is the job list, with the run time of each job:
Job 0 ( length = 200.0 )
Job 1 ( length = 200.0 )
Job 2 ( length = 200.0 )
Job 3 ( length = 200.0 )
Job 4 ( length = 200.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 200.00 secs ( DONE at 200.00 )
[ time 200 ] Run job 1 for 200.00 secs ( DONE at 400.00 )
[ time 400 ] Run job 2 for 200.00 secs ( DONE at 600.00 )
[ time 600 ] Run job 3 for 200.00 secs ( DONE at 800.00 )
[ time 800 ] Run job 4 for 200.00 secs ( DONE at 1000.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 200.00 Wait 0.00
Job 1 -- Response: 200.00 Turnaround 400.00 Wait 200.00
Job 2 -- Response: 400.00 Turnaround 600.00 Wait 400.00
Job 3 -- Response: 600.00 Turnaround 800.00 Wait 600.00
Job 4 -- Response: 800.00 Turnaround 1000.00 Wait 800.00
Average -- Response: 400.00 Turnaround 600.00 Wait 400.00
[jadhav2@csci-gnode-03 HW2]$
```

For five jobs each with length 200, both FIFO and SJF give the same result. Since all jobs have equal length, SJF schedules them in the same order as FIFO. The response times are 0, 200, 400, 600, and 800, and the turnaround times are 200, 400, 600, 800, and 1000. The average response time is 400 and the average turnaround time is 600. Therefore, FIFO and SJF produce identical results for this workload because no job is shorter than another.

2. Now do the same but with jobs of different lengths: 100, 200, and 300.

Output:

- **FIFO Scheduler**

```
tejaljadhav — jadhvt2@pceasapp961:~/HW2 — ssh jadhvt2@csegrid.ucdenver.pvt — 208x65
[jadhvt2@csci-gnode-03 HW2]$ python3 scheduler.py -p FIFO -l 100,200,300 -c
ARG policy FIFO
ARG jlist 100,200,300

Here is the job list, with the run time of each job:
Job 0 ( length = 100.0 )
Job 1 ( length = 200.0 )
Job 2 ( length = 300.0 )

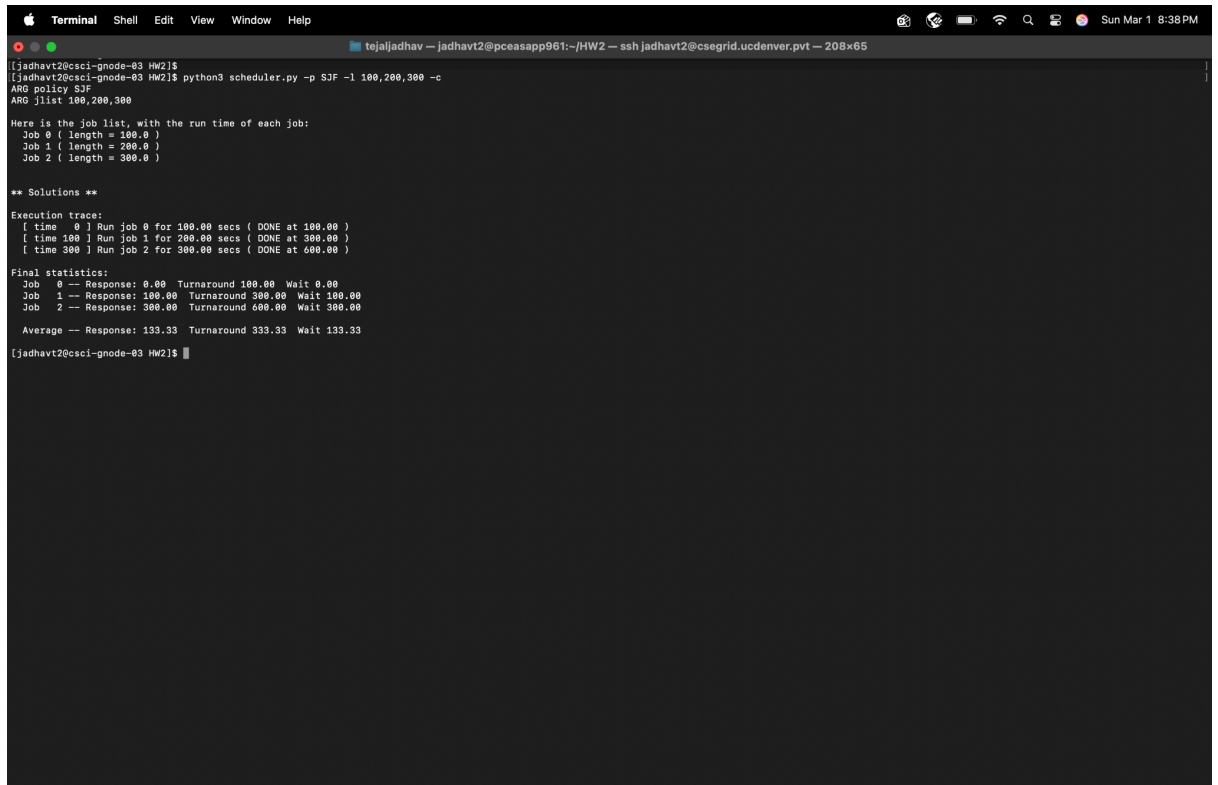
** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 100.00 secs ( DONE at 100.00 )
[ time 100 ] Run job 1 for 200.00 secs ( DONE at 300.00 )
[ time 300 ] Run job 2 for 300.00 secs ( DONE at 600.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 100.00 Wait 0.00
Job 1 -- Response: 100.00 Turnaround 300.00 Wait 100.00
Job 2 -- Response: 300.00 Turnaround 600.00 Wait 300.00
Average -- Response: 133.33 Turnaround 333.33 Wait 133.33

[jadhvt2@csci-gnode-03 HW2]$
```

- **SJF Scheduler**

A terminal window titled 'tejaljadhav — jadhvt2@pceasapp961:~/HW2 — ssh jadhvt2@csegrid.ucdenver.pvt — 208x65'. The prompt is '[jadhvt2@csci-gnode-03 HW2]\$. The user runs 'python3 scheduler.py -p SJF -l 100,200,300 -c'. The program outputs the job list, execution trace, and final statistics. The job list shows three jobs with lengths 100, 200, and 300. The execution trace shows jobs running in order. The final statistics show response times of 0, 100, and 300, and turnaround times of 100, 300, and 600. The average response time is 133.33 and the average turnaround time is 333.33.

```
[jadhvt2@csci-gnode-03 HW2]$ python3 scheduler.py -p SJF -l 100,200,300 -c
ARG policy SJF
ARG jlist 100,200,300

Here is the job list, with the run time of each job:
Job 0 ( length = 100.0 )
Job 1 ( length = 200.0 )
Job 2 ( length = 300.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 100.00 secs ( DONE at 100.00 )
[ time 100 ] Run job 1 for 200.00 secs ( DONE at 300.00 )
[ time 300 ] Run job 2 for 300.00 secs ( DONE at 600.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 100.00 Wait 0.00
Job 1 -- Response: 100.00 Turnaround 300.00 Wait 100.00
Job 2 -- Response: 300.00 Turnaround 600.00 Wait 300.00
Average -- Response: 133.33 Turnaround 333.33 Wait 133.33

[jadhvt2@csci-gnode-03 HW2]$
```

For three jobs with lengths 100, 200, and 300, both FIFO and SJF produce the same result because the jobs are already arranged in increasing order of length.

In FIFO, job 0 (100) runs first, then job 1 (200), and then job 2 (300). The response times are 0, 100, and 300. The turnaround times are 100, 300, and 600. The average response time is 133.33 and the average turnaround time is 333.33.

In SJF, the scheduler selects the shortest job first. Since the jobs are already ordered as 100, 200, and 300, the execution order does not change. Therefore, the response times and turnaround times are exactly the same as FIFO.

- Now do the same, but also with the RR scheduler and a time-slice of 1.

Output:

```
Terminal Shell Edit View Window Help
tejaljadhav — jadhav2@pceasapp961:~/HW2 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65

[jadhav2@csci-gnode-03 HW2]$
[jadhav2@csci-gnode-03 HW2]$ python3 scheduler.py -p RR -l 100,200,300 -q 1 -c
Ans policy RR
Ans jlist 100,200,300

Here is the job list, with the run time of each job:
Job 0 ( length = 100.0 )
Job 1 ( length = 200.0 )
Job 2 ( length = 300.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 1.00 secs
[ time 1 ] Run job 1 for 1.00 secs
[ time 2 ] Run job 2 for 1.00 secs
[ time 3 ] Run job 0 for 1.00 secs
[ time 4 ] Run job 1 for 1.00 secs
[ time 5 ] Run job 2 for 1.00 secs
[ time 6 ] Run job 0 for 1.00 secs
[ time 7 ] Run job 1 for 1.00 secs
[ time 8 ] Run job 2 for 1.00 secs
[ time 9 ] Run job 0 for 1.00 secs
[ time 10 ] Run job 1 for 1.00 secs
[ time 11 ] Run job 2 for 1.00 secs
[ time 12 ] Run job 0 for 1.00 secs
[ time 13 ] Run job 1 for 1.00 secs
[ time 14 ] Run job 2 for 1.00 secs
[ time 15 ] Run job 0 for 1.00 secs
[ time 16 ] Run job 1 for 1.00 secs
[ time 17 ] Run job 2 for 1.00 secs
[ time 18 ] Run job 0 for 1.00 secs
[ time 19 ] Run job 1 for 1.00 secs
[ time 20 ] Run job 2 for 1.00 secs
[ time 21 ] Run job 0 for 1.00 secs
[ time 22 ] Run job 1 for 1.00 secs
[ time 23 ] Run job 2 for 1.00 secs
[ time 24 ] Run job 0 for 1.00 secs
[ time 25 ] Run job 1 for 1.00 secs
[ time 26 ] Run job 2 for 1.00 secs
[ time 27 ] Run job 0 for 1.00 secs
[ time 28 ] Run job 1 for 1.00 secs
[ time 29 ] Run job 2 for 1.00 secs
[ time 30 ] Run job 0 for 1.00 secs
[ time 31 ] Run job 1 for 1.00 secs
[ time 32 ] Run job 2 for 1.00 secs
[ time 33 ] Run job 0 for 1.00 secs
[ time 34 ] Run job 1 for 1.00 secs
[ time 35 ] Run job 2 for 1.00 secs
[ time 36 ] Run job 0 for 1.00 secs
[ time 37 ] Run job 1 for 1.00 secs
[ time 38 ] Run job 2 for 1.00 secs
[ time 39 ] Run job 0 for 1.00 secs
[ time 40 ] Run job 1 for 1.00 secs
[ time 41 ] Run job 2 for 1.00 secs
[ time 42 ] Run job 0 for 1.00 secs
[ time 43 ] Run job 1 for 1.00 secs
[ time 44 ] Run job 2 for 1.00 secs
[ time 45 ] Run job 0 for 1.00 secs
[ time 46 ] Run job 1 for 1.00 secs
[ time 47 ] Run job 2 for 1.00 secs
[ time 48 ] Run job 0 for 1.00 secs
```

```
Terminal Shell Edit View Window Help
tejaljadhav — jadhav2@pceasapp961:~/HW2 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65

[ time 42 ] Run job 0 for 1.00 secs
[ time 43 ] Run job 1 for 1.00 secs
[ time 44 ] Run job 2 for 1.00 secs
[ time 45 ] Run job 0 for 1.00 secs
[ time 46 ] Run job 1 for 1.00 secs
[ time 47 ] Run job 2 for 1.00 secs
[ time 48 ] Run job 0 for 1.00 secs
[ time 49 ] Run job 1 for 1.00 secs
[ time 50 ] Run job 2 for 1.00 secs
[ time 51 ] Run job 0 for 1.00 secs
[ time 52 ] Run job 1 for 1.00 secs
[ time 53 ] Run job 2 for 1.00 secs
[ time 54 ] Run job 0 for 1.00 secs
[ time 55 ] Run job 1 for 1.00 secs
[ time 56 ] Run job 2 for 1.00 secs
[ time 57 ] Run job 0 for 1.00 secs
[ time 58 ] Run job 1 for 1.00 secs
[ time 59 ] Run job 2 for 1.00 secs
[ time 60 ] Run job 0 for 1.00 secs
[ time 61 ] Run job 1 for 1.00 secs
[ time 62 ] Run job 2 for 1.00 secs
[ time 63 ] Run job 0 for 1.00 secs
[ time 64 ] Run job 1 for 1.00 secs
[ time 65 ] Run job 2 for 1.00 secs
[ time 66 ] Run job 0 for 1.00 secs
[ time 67 ] Run job 1 for 1.00 secs
[ time 68 ] Run job 2 for 1.00 secs
[ time 69 ] Run job 0 for 1.00 secs
[ time 70 ] Run job 1 for 1.00 secs
[ time 71 ] Run job 2 for 1.00 secs
[ time 72 ] Run job 0 for 1.00 secs
[ time 73 ] Run job 1 for 1.00 secs
[ time 74 ] Run job 2 for 1.00 secs
[ time 75 ] Run job 0 for 1.00 secs
[ time 76 ] Run job 1 for 1.00 secs
[ time 77 ] Run job 2 for 1.00 secs
[ time 78 ] Run job 0 for 1.00 secs
[ time 79 ] Run job 1 for 1.00 secs
[ time 80 ] Run job 2 for 1.00 secs
[ time 81 ] Run job 0 for 1.00 secs
[ time 82 ] Run job 1 for 1.00 secs
[ time 83 ] Run job 2 for 1.00 secs
[ time 84 ] Run job 0 for 1.00 secs
[ time 85 ] Run job 1 for 1.00 secs
[ time 86 ] Run job 2 for 1.00 secs
[ time 87 ] Run job 0 for 1.00 secs
[ time 88 ] Run job 1 for 1.00 secs
[ time 89 ] Run job 2 for 1.00 secs
[ time 90 ] Run job 0 for 1.00 secs
[ time 91 ] Run job 1 for 1.00 secs
[ time 92 ] Run job 2 for 1.00 secs
[ time 93 ] Run job 0 for 1.00 secs
[ time 94 ] Run job 1 for 1.00 secs
[ time 95 ] Run job 2 for 1.00 secs
[ time 96 ] Run job 0 for 1.00 secs
[ time 97 ] Run job 1 for 1.00 secs
[ time 98 ] Run job 2 for 1.00 secs
[ time 99 ] Run job 0 for 1.00 secs
[ time 100 ] Run job 1 for 1.00 secs
[ time 101 ] Run job 2 for 1.00 secs
[ time 102 ] Run job 0 for 1.00 secs
[ time 103 ] Run job 1 for 1.00 secs
[ time 104 ] Run job 2 for 1.00 secs
```



```
tejaljadhav — jadhav2@pceasapp961:~/HW2 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[ time 546 ] Run job 2 for 1.00 secs
[ time 547 ] Run job 2 for 1.00 secs
[ time 548 ] Run job 2 for 1.00 secs
[ time 549 ] Run job 2 for 1.00 secs
[ time 550 ] Run job 2 for 1.00 secs
[ time 551 ] Run job 2 for 1.00 secs
[ time 552 ] Run job 2 for 1.00 secs
[ time 553 ] Run job 2 for 1.00 secs
[ time 554 ] Run job 2 for 1.00 secs
[ time 555 ] Run job 2 for 1.00 secs
[ time 556 ] Run job 2 for 1.00 secs
[ time 557 ] Run job 2 for 1.00 secs
[ time 558 ] Run job 2 for 1.00 secs
[ time 559 ] Run job 2 for 1.00 secs
[ time 560 ] Run job 2 for 1.00 secs
[ time 561 ] Run job 2 for 1.00 secs
[ time 562 ] Run job 2 for 1.00 secs
[ time 563 ] Run job 2 for 1.00 secs
[ time 564 ] Run job 2 for 1.00 secs
[ time 565 ] Run job 2 for 1.00 secs
[ time 566 ] Run job 2 for 1.00 secs
[ time 567 ] Run job 2 for 1.00 secs
[ time 568 ] Run job 2 for 1.00 secs
[ time 569 ] Run job 2 for 1.00 secs
[ time 570 ] Run job 2 for 1.00 secs
[ time 571 ] Run job 2 for 1.00 secs
[ time 572 ] Run job 2 for 1.00 secs
[ time 573 ] Run job 2 for 1.00 secs
[ time 574 ] Run job 2 for 1.00 secs
[ time 575 ] Run job 2 for 1.00 secs
[ time 576 ] Run job 2 for 1.00 secs
[ time 577 ] Run job 2 for 1.00 secs
[ time 578 ] Run job 2 for 1.00 secs
[ time 579 ] Run job 2 for 1.00 secs
[ time 580 ] Run job 2 for 1.00 secs
[ time 581 ] Run job 2 for 1.00 secs
[ time 582 ] Run job 2 for 1.00 secs
[ time 583 ] Run job 2 for 1.00 secs
[ time 584 ] Run job 2 for 1.00 secs
[ time 585 ] Run job 2 for 1.00 secs
[ time 586 ] Run job 2 for 1.00 secs
[ time 587 ] Run job 2 for 1.00 secs
[ time 588 ] Run job 2 for 1.00 secs
[ time 589 ] Run job 2 for 1.00 secs
[ time 590 ] Run job 2 for 1.00 secs
[ time 591 ] Run job 2 for 1.00 secs
[ time 592 ] Run job 2 for 1.00 secs
[ time 593 ] Run job 2 for 1.00 secs
[ time 594 ] Run job 2 for 1.00 secs
[ time 595 ] Run job 2 for 1.00 secs
[ time 596 ] Run job 2 for 1.00 secs
[ time 597 ] Run job 2 for 1.00 secs
[ time 598 ] Run job 2 for 1.00 secs
[ time 599 ] Run job 2 for 1.00 secs ( DONE at 600.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 298.00 Wait 198.00
Job 1 -- Response: 1.00 Turnaround 499.00 Wait 299.00
Job 2 -- Response: 2.00 Turnaround 600.00 Wait 300.00

Average -- Response: 1.00 Turnaround 465.67 Wait 265.67

[jadhav2@csci-gnode-03 HW2]$
```

With Round Robin and time slice 1, each job runs for 1 unit and then the CPU switches to the next job. This keeps going again and again until the jobs finish.

From the output:

Job 0 → Response time = 0, Turnaround time = 298

Job 1 → Response time = 1, Turnaround time = 499

Job 2 → Response time = 2, Turnaround time = 600

Average response time = 1.00

Average turnaround time = 465.67

Since the time slice is very small, all jobs get CPU quickly, so response time is very low. But because the CPU keeps switching between jobs many times, the turnaround time becomes higher compared to FIFO and SJF.

4. For what types of workloads does SJF deliver the same turnaround times as FIFO?

Answer:

SJF gives the same turnaround time as FIFO for workloads where the jobs are already arranged from shortest to longest. In this type of workload, FIFO runs the jobs in the same order that SJF would choose, so both schedulers produce the same turnaround times. SJF also gives the same result for workloads where all jobs have equal lengths, because the execution order does not affect the turnaround time when every job takes the same amount of time.

5. For what types of workloads and quantum lengths does SJF deliver the same response times as RR?

Answer:

SJF gives the same response times as RR when the workload is such that RR behaves like a non-preemptive scheduler. This happens when the time quantum is large enough that each job finishes in one turn without being interrupted. In that case, RR works like FIFO. If the workload is already arranged from shortest to longest, then FIFO and SJF run the jobs in the same order, so the response times are the same.

It can also happen in workloads where all jobs have equal lengths, because the order of execution does not change the response time in that situation.

6. What happens to response time with SJF as job lengths increase? Can you use the simulator to demonstrate the trend?

Answer:

With SJF, response time increases as job lengths increase. In SJF, each job must wait for all shorter jobs to finish before it starts. If the job lengths become larger, then the waiting time before later jobs start also becomes larger. So response time grows as job lengths grow.

To demonstrate this using the simulator:

- First run with smaller job lengths:

```
tejal@jadhav:~$ python3 scheduler.py -p SJF -l 5,10,15,20 -c
ARG policy SJF
ARG jlist 5,10,15,20

Here is the job list, with the run time of each job:
Job 0 ( length = 5.0 )
Job 1 ( length = 10.0 )
Job 2 ( length = 15.0 )
Job 3 ( length = 20.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 5.00 secs ( DONE at 5.00 )
[ time 5 ] Run job 1 for 10.00 secs ( DONE at 15.00 )
[ time 15 ] Run job 2 for 15.00 secs ( DONE at 30.00 )
[ time 30 ] Run job 3 for 20.00 secs ( DONE at 50.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 5.00 Wait 0.00
Job 1 -- Response: 5.00 Turnaround 15.00 Wait 5.00
Job 2 -- Response: 15.00 Turnaround 30.00 Wait 15.00
Job 3 -- Response: 30.00 Turnaround 50.00 Wait 30.00

Average -- Response: 12.50 Turnaround 25.00 Wait 12.50

[jadhav2@csci-gnode-03 HW2]$
```

Response times:

0, 5, 15, 30

Average response time = 12.50

- Run with larger job lengths:

```
tejal@jadhav:~$ python3 scheduler.py -p SJF -l 50,100,150,200 -c
ARG policy SJF
ARG jlist 50,100,150,200

Here is the job list, with the run time of each job:
Job 0 ( length = 50.0 )
Job 1 ( length = 100.0 )
Job 2 ( length = 150.0 )
Job 3 ( length = 200.0 )

** Solutions **

Execution trace:
[ time 0 ] Run job 0 for 50.00 secs ( DONE at 50.00 )
[ time 50 ] Run job 1 for 100.00 secs ( DONE at 150.00 )
[ time 150 ] Run job 2 for 150.00 secs ( DONE at 300.00 )
[ time 300 ] Run job 3 for 200.00 secs ( DONE at 500.00 )

Final statistics:
Job 0 -- Response: 0.00 Turnaround 50.00 Wait 0.00
Job 1 -- Response: 50.00 Turnaround 150.00 Wait 50.00
Job 2 -- Response: 150.00 Turnaround 300.00 Wait 150.00
Job 3 -- Response: 300.00 Turnaround 500.00 Wait 300.00

Average -- Response: 125.00 Turnaround 250.00 Wait 125.00

[jadhav2@csci-gnode-03 HW2]$
```

Response times:

0, 50, 150, 300

Average response time = 125.00

Comparing both runs shows that when the job lengths increase, the response times increase significantly. This demonstrates the trend clearly.

7. What happens to response time with RR as quantum lengths increase? Can you write an equation that gives the worst-case response time, given N jobs?

Answer:

As the quantum length increases in Round Robin scheduling, the response time increases. When the time slice is small, the scheduler switches between jobs quickly, so each job gets CPU time soon after it arrives. This improves performance under the response-time metric. However, as the quantum becomes larger, each job runs longer before the next job gets a turn. Because of this, jobs later in the queue must wait more time before starting. If the quantum becomes very large, Round Robin behaves like FIFO, and the response time becomes much worse.

In the worst case, if N jobs arrive at the same time, the last job must wait for the other $N - 1$ jobs to each run once before it gets CPU time. Since each job runs for q time units, the worst-case response time is:

$$R_{worst} = (N-1) \times q$$

This shows that response time increases as the quantum length and the number of jobs increase.